

XGBoost গণিত: Objective → Taylor Expansion → Leaf Weight → Split Gain

ভূমিকা

এই ডকুমেন্টে XGBoost-এর objective function থেকে শুরু করে Taylor expansion, leaf weight, optimal objective এবং split gain পর্যন্ত পুরো chain লাইন বাই লাইন সহজ গণিত দিয়ে দেখানো হয়েছে।

১. শুরু: XGBoost কী optimize করতে চায়?

ধরি, এখন আমরা m -তম tree তৈরি করছি।

আগের সব tree-এর prediction:

$$F_{m-1}(x_i)$$

নতুন tree যোগ করবে:

$$h_m(x_i)$$

তাহলে নতুন prediction হবে:

$$F_m(x_i) = F_{m-1}(x_i) + h_m(x_i)$$

XGBoost-এর objective:

$$Obj^{(m)} = \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + h_m(x_i)) + \Omega(h_m)$$

এখানে:

- y_i = আসল target
- $F_{m-1}(x_i)$ = আগের model-এর prediction
- $h_m(x_i)$ = নতুন tree-এর contribution
- L = loss function
- $\Omega(h_m)$ = tree complexity penalty

২. মূল সমস্যা কোথায়?

আমাদের আছে:

$$L(y_i, F_{m-1}(x_i) + h_m(x_i))$$

এই loss function-এর মধ্যে $h_m(x_i)$ টুকে আছে। অনেক loss function সরাসরি সহজে minimize করা যায় না। তাই XGBoost এখানে Second-order Taylor approximation ব্যবহার করে।

৩. Taylor Expansion কোথা থেকে আসে?

সাধারণ Taylor expansion:

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x + \frac{1}{2}f''(x)(\Delta x)^2$$

এখন এটাকে XGBoost-এর ক্ষেত্রে বসাই। ধরি:

$$x = F_{m-1}(x_i)$$

এবং:

$$\Delta x = h_m(x_i)$$

তাহলে:

$$L(y_i, F_{m-1}(x_i) + h_m(x_i))$$

approximately হবে:

$$L(y_i, F_{m-1}(x_i)) + g_i h_m(x_i) + \frac{1}{2} f_i h_m(x_i)^2$$

এখানে সাধারণত f_i -এর বদলে h_i বা H_i -ও লেখা হয়। তুমি যে notation দিয়েছ:

$$g_i = \frac{\partial L}{\partial F_{m-1}(x_i)}$$

অর্থাৎ first derivative / gradient। আর:

$$f_i = \frac{\partial^2 L}{\partial F_{m-1}(x_i)^2}$$

অর্থাৎ second derivative / Hessian।

8. পুরো objective-এ Taylor expansion বসাই

আগের objective ছিল:

$$Obj^{(m)} = \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + h_m(x_i)) + \Omega(h_m)$$

এখন loss-এর জায়গায় Taylor approximation বসাই:

$$Obj^{(m)} \approx \sum_{i=1}^n \left[L(y_i, F_{m-1}(x_i)) + g_i h_m(x_i) + \frac{1}{2} f_i h_m(x_i)^2 \right] + \Omega(h_m)$$

অর্থাৎ:

$$Obj^{(m)} \approx \sum_{i=1}^n \left[L_i + g_i h_m(x_i) + \frac{1}{2} f_i h_m(x_i)^2 \right] + \Omega(h_m)$$

এখানে:

$$L_i = L(y_i, F_{m-1}(x_i))$$

৫. প্রথম term-টা কেন বাদ দেওয়া যায়?

দেখো:

$$L(y_i, F_{m-1}(x_i))$$

এটা আগের model-এর loss। আমরা এখন শুধু নতুন tree h_m optimize করছি। এই term-এর মধ্যে h_m নেই। অর্থাৎ নতুন tree যেভাবেই পরিবর্তন করি:

$$L(y_i, F_{m-1}(x_i))$$

পরিবর্তন হবে না। তাই optimization-এর সময় এটাকে constant ধরে বাদ দিতে পারি। ফলে:

$$\widehat{Obj} = \sum_{i=1}^n \left[g_i h_m(x_i) + \frac{1}{2} f_i h_m(x_i)^2 \right] + \Omega(h_m)$$

এখন আমাদের কাজ হলো এই expression minimize করা।

৬. এবার Tree-এর ভিতরে যাই

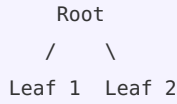
একটি decision tree-তে প্রত্যেক sample একটি leaf-এ যায়। ধরি, tree-তে মোট T টি leaf আছে। প্রত্যেক leaf-এর prediction বা score: w_j

যদি sample i , leaf j -তে যায়:

$$h_m(x_i) = w_j$$

অর্থাৎ একই leaf-এর সব sample একই output পাবে।

উদাহরণ



Leaf 1-এর score: w_1 । Leaf 2-এর score: w_2 । যদি sample x_1, x_2 Leaf 1-এ যায়:

$$h(x_1) = w_1 \quad h(x_2) = w_1$$

৭. এবার regularization

XGBoost tree-এর complexity কমাতে ব্যবহার করে:

$$\Omega(h) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

এখানে γT — প্রতিটি leaf-এর জন্য penalty। বেশি leaf হলে:

$$T \uparrow \Rightarrow \gamma T \uparrow$$

অর্থাৎ অপ্রয়োজনীয় split কমানোর চেষ্টা করে। λ — Leaf weight w_j -কে খুব বড় হওয়া থেকে আটকায়।

৮. Objective-কে leaf অনুযায়ী সাজাই

আমাদের objective:

$$\widehat{Obj} = \sum_{i=1}^n \left[g_i h(x_i) + \frac{1}{2} f_i h(x_i)^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

ধরি I_j হলো সেই sample-গুলোর set যেগুলো leaf j -তে গেছে। সেই leaf-এর জন্য $h(x_i) = w_j$ । তাই:

$$\sum_{i \in I_j} \left[g_i w_j + \frac{1}{2} f_i w_j^2 \right]$$

এখন w_j বাইরে বের করতে পারি:

$$= w_j \sum_{i \in I_j} g_i + \frac{1}{2} w_j^2 \sum_{i \in I_j} f_i$$

এখন define করি:

$$G_j = \sum_{i \in I_j} g_i$$

এবং:

$$H_j = \sum_{i \in I_j} f_i$$

তাহলে:

$$G_j w_j + \frac{1}{2} H_j w_j^2$$

Regularization-সহ $\frac{1}{2} \lambda w_j^2$ যোগ করলে:

$$G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2$$

সুতরাং পুরো objective:

$$\widehat{Obj} = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T$$

এটাই leaf weight বের করার মূল equation।

৯. এবার leaf-এর best weight w_j^* বের করি

একটি leaf-এর অংশ দেখি:

$$Obj_j = G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2$$

আমরা চাই w_j এর এমন value, যাতে objective minimum হয়। তাই derivative নিই:

$$\frac{\partial Obj_j}{\partial w_j} = \frac{\partial}{\partial w_j} \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right]$$

প্রথম term:

$$\frac{\partial}{\partial w_j} (G_j w_j) = G_j$$

কারণ G_j constant। দ্বিতীয় term:

$$\frac{\partial}{\partial w_j} \left[\frac{1}{2} (H_j + \lambda) w_j^2 \right] = \frac{1}{2} (H_j + \lambda) \times 2 w_j = (H_j + \lambda) w_j$$

তাহলে:

$$\frac{\partial Obj_j}{\partial w_j} = G_j + (H_j + \lambda) w_j$$

Minimum পাওয়ার জন্য derivative = 0:

$$G_j + (H_j + \lambda)w_j = 0$$

এখন w_j বের করি:

$$(H_j + \lambda)w_j = -G_j$$

অতএব:

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

এটাই XGBoost-এর optimal leaf weight।

১০. এখন Obj^* কোথা থেকে আসে?

আমাদের leaf objective ছিল:

$$Obj_j = G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2$$

এখন best weight $w_j^* = -\frac{G_j}{H_j + \lambda}$ — এটা objective-এ বসাই।

প্রথম term

$$G_j w_j^* = G_j \left(-\frac{G_j}{H_j + \lambda} \right) = -\frac{G_j^2}{H_j + \lambda}$$

দ্বিতীয় term

$$\frac{1}{2}(H_j + \lambda)(w_j^*)^2$$

প্রথমে square করি:

$$(w_j^*)^2 = \left(-\frac{G_j}{H_j + \lambda} \right)^2$$

Minus square করলে plus:

$$= \frac{G_j^2}{(H_j + \lambda)^2}$$

এখন বসাই:

$$\frac{1}{2}(H_j + \lambda) \frac{G_j^2}{(H_j + \lambda)^2}$$

একটা denominator cancel হবে:

$$= \frac{1}{2} \frac{G_j^2}{H_j + \lambda}$$

এখন দুইটা যোগ করি

প্রথম term: $-\frac{G_j^2}{H_j + \lambda}$ দ্বিতীয় term: $+\frac{1}{2} \frac{G_j^2}{H_j + \lambda}$

অর্থাৎ:

$$-\frac{G_j^2}{H_j + \lambda} + \frac{1}{2} \frac{G_j^2}{H_j + \lambda} = -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}$$

তাই একটি leaf-এর optimal objective:

$$Obj_j^* = -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}$$

সব leaf-এর জন্য:

$$Obj^* = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

তুমি যে equation লিখেছ, এভাবেই এসেছে।

১১. এখন আসল মজার অংশ: Gain

XGBoost split করার আগে দেখে — এই node-টাকে দুই ভাগ করলে objective improve হবে কি না?

Split করার আগে

ধরি পুরো node-এ:

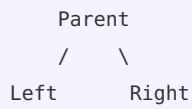
$$G = G_L + G_R \quad H = H_L + H_R$$

Split করার আগে একটাই leaf। তার optimal score:

$$Score_{before} = -\frac{1}{2} \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}$$

১২. Split করার পরে

এখন দুটি leaf:



Left leaf-এর score:

$$-\frac{1}{2} \frac{G_L^2}{H_L + \lambda}$$

Right leaf-এর score:

$$-\frac{1}{2} \frac{G_R^2}{H_R + \lambda}$$

দুইটা মিলিয়ে:

$$Score_{after} = -\frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} \right]$$

১৩. Gain মানে কী?

আমরা objective minimize করছি। তাই improvement:

$$Gain = Obj_{before} - Obj_{after}$$

কারণ split করার পরে যদি objective কমে যায়, তাহলে $Obj_{before} > Obj_{after}$ — তাহলে improvement positive হবে।

Formula বসাই

$$Gain = \left[-\frac{1}{2} \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \left[-\frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} \right) \right]$$

দ্বিতীয় bracket-এর minus খুললে:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right]$$

এখন split করার কারণে নতুন একটি extra leaf হয়েছে। তাই complexity penalty: $-\gamma$ । ফলে:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

এটাই XGBoost-এর famous split gain formula।

১৪. একটি ছোট নোট formula নিয়ে

সঠিক standard formula হলো উপরেরটাই। তবে যদি কোথাও ভুলবশত এমন লেখা থাকে:

$$H_L + H_R + \lambda(G_L + G_R)^2$$

তাহলে fraction-এর bracket/format ঠিকভাবে লেখা হয়নি। সঠিকটা হলো:

$$\frac{(G_L + G_R)^2}{H_L + H_R + \lambda}$$

১৫. একদম সংক্ষেপে পুরো flow

Step 1: Original objective

$$Obj = \sum_i L(y_i, F_{old}(x_i) + h(x_i)) + \Omega(h)$$

↓ Taylor expansion

Step 2: Approximation

$$Obj \approx \sum_i \left[g_i h(x_i) + \frac{1}{2} H_i h(x_i)^2 \right] + \Omega(h)$$

↓ Samples-কে leaf অনুযায়ী group করি

Step 3: Leaf statistics

$$G_j = \sum_{i \in I_j} g_i \quad H_j = \sum_{i \in I_j} H_i$$

↓

Step 4: Leaf objective

$$G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2$$

↓ Derivative = 0

Step 5: Best leaf weight

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

↓ w_j^* objective-এ বসাই

Step 6: Optimal tree objective

$$Obj^* = -\frac{1}{2} \sum_j \frac{G_j^2}{H_j + \lambda} + \gamma T$$

↓ Split before vs after compare করি

Step 7: Gain

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

সবচেয়ে সহজ intuition

XGBoost প্রতিটি সম্ভাব্য split-এর জন্য মূলত জিজ্ঞেস করে: এই split করার পরে Left এবং Right group আলাদা করে prediction দেওয়া কি আগের একসাথে prediction দেওয়ার চেয়ে ভালো?

যদি $Gain > 0$ তাহলে split লাভজনক। আর যদি $Gain \leq 0$ তাহলে সাধারণভাবে সেই split নেওয়ার কোনো লাভ নেই।

মূল chainটি শুধু এই:

$Loss \rightarrow Taylor Approximation \rightarrow G, H \rightarrow Leaf Weight \rightarrow Optimal Score \rightarrow Split Gain$

reg_alpha (L1 Regularization) বিস্তারিত ব্যাখ্যা

reg_alpha বুঝতে হলে আগে জানতে হবে reg_lambda কী করছিল। আগের XGBoost objective-এ একটি leaf-এর জন্য ছিল:

$$Obj_j = G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2$$

এখানে:

$$G_j = \sum_{i \in I_j} g_i \quad H_j = \sum_{i \in I_j} h_i$$

এবং λ হলো reg_lambda।

১. reg_alpha কী?

XGBoost-এ:

- reg_lambda = L2 regularization
- reg_alpha = L1 regularization

ধরি $\alpha = \text{reg_alpha}$ । তাহলে leaf weight-এর উপর একটি নতুন penalty যোগ হবে:

$$\alpha |w_j|$$

তাই নতুন leaf objective:

$$Obj_j = G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 + \alpha |w_j|$$

এখন এখানেই মূল পার্থক্য। আগে ছিল $\frac{1}{2} \lambda w_j^2$ । এখন যোগ হয়েছে $\alpha |w_j|$ । এই absolute value term-এর কারণেই reg_alpha কোনো leaf-এর weight একদম 0 করে দিতে পারে।

২. কেন $|w|$ এত গুরুত্বপূর্ণ?

ধরি $w = 5$ তাহলে $|w| = 5$ । আবার $w = -5$ তাহলে $|w| = 5$ । আর $w = 0$ তাহলে $|w| = 0$ । অর্থাৎ L1 penalty weight-কে 0-এর দিকে টেনে আনে। $\alpha |w|$ — যত বড় α , weight non-zero রাখার penalty তত বেশি।

৩. এখন math করি

আমাদের objective:

$$Obj(w) = Gw + \frac{1}{2} (H + \lambda) w^2 + \alpha |w|$$

এখানে সহজ করার জন্য j বাদ দিলাম। কারণ $|w|$ আছে, তাই আমরা একবারে derivative নিতে পারি না। আমাদের দুইটা case দেখতে হবে।

Case 1: $w > 0$

যদি $w > 0$ তাহলে $|w| = w$ । সুতরাং:

$$Obj(w) = Gw + \frac{1}{2} (H + \lambda) w^2 + \alpha w$$

এখন derivative নিই:

$$\frac{dObj}{dw} = G + (H + \lambda)w + \alpha$$



Minimum-এর জন্য:

$$G + (H + \lambda)w + \alpha = 0$$

তাহলে $(H + \lambda)w = -G - \alpha$ । সুতরাং:

$$w = -\frac{G + \alpha}{H + \lambda}$$

কিন্তু আমরা শুরুতেই ধরে নিয়েছি $w > 0$ । তাই numerator এমন হতে হবে যাতে result positive হয়।

Case 2: $w < 0$

যদি $w < 0$ তাহলে $|w| = -w$ । তাই:

$$Obj(w) = Gw + \frac{1}{2}(H + \lambda)w^2 - \alpha w$$

Derivative:

$$\frac{dObj}{dw} = G + (H + \lambda)w - \alpha$$

Minimum-এর জন্য $G + (H + \lambda)w - \alpha = 0$ । তাহলে $(H + \lambda)w = -G + \alpha$ । অতএব:

$$w = -\frac{G - \alpha}{H + \lambda}$$

৪. তাহলে $w = 0$ কখন হবে?

এটাই সবচেয়ে গুরুত্বপূর্ণ অংশ। আগে reg_alpha ছাড়া:

$$w^* = -\frac{G}{H + \lambda}$$

অর্থাৎ যতক্ষণ $G \neq 0$ সাধারণত weight zero হবে না। কিন্তু reg_alpha থাকলে এমন হতে পারে যে:

$$|G| \leq \alpha$$

তখন:

$$w^* = 0$$

এটাই হলো reg_alpha-এর sparsity effect।

৫. কেন $|G| \leq \alpha$ হলে weight 0?

ধরি $G = 3$ এবং $\alpha = 5$ । এখন $w > 0$ case দেখি:

$$w = -\frac{G + \alpha}{H + \lambda} = -\frac{3 + 5}{H + \lambda}$$

এটা negative। কিন্তু আমরা এই case-এ ধরেছিলাম $w > 0$ । তাই এই solution valid না।

এখন $w < 0$ case:

$$w = -\frac{G - \alpha}{H + \lambda} = -\frac{3 - 5}{H + \lambda} = \frac{2}{H + \lambda}$$

এটা positive। কিন্তু আমরা ধরেছিলাম $w < 0$ । এটাও valid না।

তাহলে positive side-এও valid solution নেই, negative side-এও valid solution নেই। তাই minimum হবে মাঝখানে:

$$w = 0$$

৬. তিনটি situation

ধরি $\alpha = 5$ ।

Situation A: $G > 5$

ধরি $G = 10$ । তাহলে $G > \alpha$ । Weight হবে negative:

$$w^* = -\frac{G - \alpha}{H + \lambda}$$

অর্থাৎ:

$$w^* = -\frac{10 - 5}{H + \lambda} = -\frac{5}{H + \lambda}$$

আগে alpha ছাড়া ছিল $-\frac{10}{H + \lambda}$ । এখন হয়েছে $-\frac{5}{H + \lambda}$ । অর্থাৎ reg_alpha weight-কে ছোট করে দিয়েছে।

Situation B: $G < -5$

ধরি $G = -10$ । তাহলে $G < -\alpha$ । Weight positive হবে:

$$w^* = -\frac{G + \alpha}{H + \lambda}$$

তাই:

$$w^* = -\frac{-10 + 5}{H + \lambda} = \frac{5}{H + \lambda}$$

আবার weight ছোট হয়ে গেছে।

Situation C: $-5 \leq G \leq 5$

ধরি $G = 3$ এবং $\alpha = 5$ । তাহলে $|G| < \alpha$ । সুতরাং:

$$w^* = 0$$

এই leaf-এর prediction contribution $h(x) = 0$ । অর্থাৎ এই leaf নতুন tree-এর prediction-এ কোনো পরিবর্তন আনছে না।

৭. পুরো formula একসাথে

reg_alpha সহ optimal leaf weight:

$$w_j^* = \begin{cases} -\frac{G_j + \alpha}{H_j + \lambda}, & G_j < -\alpha \\ 0, & |G_j| \leq \alpha \\ -\frac{G_j - \alpha}{H_j + \lambda}, & G_j > \alpha \end{cases}$$

এটাকে compactভাবে লেখা যায়:

$$w_j^* = -\frac{S(G_j, \alpha)}{H_j + \lambda}$$

যেখানে S হলো soft thresholding function:

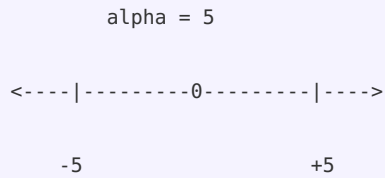
$$S(G, \alpha) = \begin{cases} G - \alpha, & G > \alpha \\ 0, & |G| \leq \alpha \\ G + \alpha, & G < -\alpha \end{cases}$$

৮. Soft Thresholding সহজভাবে বুঝি

ধরি $\alpha = 5$ । তাহলে:

G	$S(G, \alpha)$
-10	-10+5=-5
-7	-7+5=-2
-5	0
-3	0
0	0
3	0
5	0
7	7-5=2
10	10-5=5

অর্থাৎ:



এই range-এর ভিতরে $|G| \leq \alpha$ হলে weight = 0।

৯. তাহলে leaf discard কীভাবে হয়?

এখানে একটু গুরুত্বপূর্ণ clarification আছে। reg_alpha mathematically প্রথমে leaf weight-কে zero করতে পারে। অর্থাৎ $|G_j| \leq \alpha$ হলে $w_j = 0$ । তখন সেই leaf-এর output 0। অর্থাৎ tree-এর সেই অংশ prediction-এর জন্য কোনো contribution দিচ্ছে না।

Practical tree building-এর সময় XGBoost split-এর gain হিসাব করার সময়ও L1 regularization বিবেচনা করে। যদি split যথেষ্ট improvement না দেয়, তাহলে সেই split গ্রহণ করা হয় না। ফলে tree ছোট হতে পারে এবং কিছু potential leaf তৈরি হওয়ার আগেই বাদ পড়ে।

১০. reg_lambda vs reg_alpha

শুধু reg_lambda

$$w^* = -\frac{G}{H + \lambda}$$

এখানে λ বাড়লে $|w|$ ছোট হয়। কিন্তু সাধারণত $w \neq 0$ ।

reg_alpha

$$w^* = -\frac{S(G, \alpha)}{H + \lambda}$$

এখানে যদি $|G| \leq \alpha$ তাহলে:

$$w = 0$$

অর্থাৎ reg_alpha ছোট/দুর্বল leaf contribution পুরোপুরি eliminate করতে পারে।

১১. Numerical Example

ধরি $G = 8$, $H = 10$, $\lambda = 2$, $\alpha = 5$ ।

Alpha ছাড়া

$$w^* = -\frac{G}{H + \lambda} = -\frac{8}{10 + 2} = -\frac{8}{12} = -0.667$$

Alpha সহ

কারণ $G > \alpha$, তাই:

$$w^* = -\frac{G - \alpha}{H + \lambda} = -\frac{8 - 5}{10 + 2} = -\frac{3}{12} = -0.25$$

দেখো: $-0.667 \rightarrow -0.25$ । reg_alpha weight অনেক কমিয়ে দিয়েছে।

আরেকটি Example: Leaf zero হয়ে গেল

ধরি $G = 3$, $H = 10$, $\lambda = 2$, $\alpha = 5$ । কারণ $|G| = 3$ এবং $3 < 5$, তাই:

$$w^* = 0$$

অর্থাৎ:

আগে:
 Leaf
 |
 └ prediction = কিছু value

reg_alpha প্রয়োগের পরে:
 Leaf
 |
 └ prediction = 0

সবচেয়ে গুরুত্বপূর্ণ intuition

G হলো ওই leaf-এর samples-এর মোট gradient signal। XGBoost জিজ্ঞেস করে: "এই leaf-এর prediction পরিবর্তন করার মতো যথেষ্ট strong signal আছে কি?" reg_alpha বলে: "Signal ছোট হলে আমি leaf-এর weight zero করে দেব।"

গাণিতিকভাবে:

$$|G_j| \leq \alpha \Rightarrow w_j^* = 0$$

আর:

$$|G_j| > \alpha \Rightarrow \text{leaf gets a nonzero weight}$$

সুতরাং খুব সংক্ষেপে:

`reg_lambda` → weight shrink করে

`reg_alpha` → weight shrink করে, এবং কিছু ক্ষেত্রে exactly 0 করে

এটাই `reg_alpha`-এর মূল শক্তি: weak gradient signal-যুক্ত leaf/split-এর contribution বাদ দিয়ে tree-কে আরও sparse ও simple করা।